

Dependent Type Theory, an Introduction

CLAUDIO BASTIANI-FONCK

1. Introduction

Proof assistant softwares such as Lean are rapidly gaining popularity. It is possible that in the future these softwares might play a somewhat nontrivial role in how mathematics is done. The following is a conceptual introduction to the basic concepts in dependent type theory. The purpose is to give some broad intuition for what these softwares are actually doing under the hood in order to verify that a formal proof is correct.

2. Curry–Howard

The Curry–Howard correspondence states essentially that there is a correspondence:

$$\text{Mathematical Proofs} \longleftrightarrow \text{Computer Programs}$$

This is not very intuitive, so we will give an example.

Consider the statement that $f(x) = x^2$ is continuous at $x = 2$. If you’ve taken analysis or the 160s, you’ll know that this is a statement about epsilons and deltas:

$$\forall \varepsilon \in \mathbb{R}_{>0} \quad \exists \delta \in \mathbb{R}_{>0} \quad |x - 2| < \delta \implies |x^2 - 4| < \varepsilon \quad (1)$$

The ordinary mathematical proof is the following.

Fix $\varepsilon > 0$ and choose:

$$\delta := \min\left(1, \frac{\varepsilon}{5}\right)$$

Both 1 and $\varepsilon/5$ are positive, so $\delta > 0$. Suppose $|x - 2| < \delta$. Since $\delta \leq 1$:

$$|x + 2| = |(x - 2) + 4| \leq |x - 2| + 4 < \delta + 4 \leq 5$$

Also $\delta \leq \varepsilon/5$. Therefore:

$$|x^2 - 4| = |x - 2| |x + 2| < 5\delta \leq \varepsilon$$

This proves (1).

Now we will see how the mathematical proof translates to a computer program, and moreover how the statement is translated into a type.

2.1. Types

In dependent type theory, every object has a *type*. Even types have types. We write

$$a : A$$

to express that a has type A . A function which maps an element of type A to something of type B has type $A \longrightarrow B$. The upshot is that a mathematical proposition can be expressed as some type P . A proof of that proposition is a term p of that type, written as $p : P$.

As a warning, it is a common mistake to think of a type as some kind of set, and read $a : A$ as somehow analogous to $a \in A$. This is the wrong intuition for the following reason:

In Lean, every object (including a type) has a *unique* type

so if types were really sets, they would never intersect. The correct intuition is much closer to computer science or category theory. That is, the point of a type is that there is a (hopefully fast) algorithm that can determine the exact type of any expression in our formal system in finite steps. In this sense, the type of something expresses *exactly* what kind of data it is, something the computer can keep track of. For example, one cannot consider a group which happens to be abelian as an abelian group, without first mapping it to a corresponding object whose type is abelian group.

There are some versions of type theory that break the above claims, however since we are mainly concerned with Lean, one should keep the above mental model in mind.

2.2. The type of a proof

As above, we consider the proposition that $|x^2 - 4| < \epsilon$ as a type, and a term of this type is a proof that $|x^2 - 4| < \epsilon$. We express the implication that

$$|x - 2| < \delta \implies |x^2 - 4| < \epsilon \quad (2)$$

as a *function* which takes a proof of $|x - 2| < \delta$, and returns a proof that $|x^2 - 4| < \epsilon$. The type of such a function is simply:

$$|x - 2| < \delta \longrightarrow |x^2 - 4| < \epsilon$$

However this type only expresses this implication for some given x . In order to express that

$$\forall x \quad |x - 2| < \delta \implies |x^2 - 4| < \epsilon \quad (3)$$

we need a general function that takes an x and return a proof that Equation (3) is satisfied for that x . The type signature of such a function is written as:

$$\prod_{x:\mathbb{R}} |x - 2| < \delta \longrightarrow |x^2 - 4| < \epsilon$$

The $\prod$ expresses that the type of the output depends on the specific input of the function. In normal CS or category theory, functions map from a fixed type to another fixed type. Dependent function types break this restriction, and can be vaguely thought of as some equivalent to $\forall$.

To prove the theorem, we need a function which takes epsilons, and maps them to deltas which satisfy Equation (3). Under Curry–Howard, the type of such a function is.

$$\prod_{\epsilon:\mathbb{R}_{>0}} \sum_{\delta:\mathbb{R}_{>0}} \prod_{x:\mathbb{R}} (|x - 2| < \delta \longrightarrow |x^2 - 4| < \epsilon) \quad (4)$$

Read it from left to right:

1. The $\prod_{\epsilon}$ means the program is a function that receives a positive real number ϵ as input.
2. The $\sum_{\delta}$ means the function returns a dependent pair. Its first component is a chosen positive real number δ .
3. The final $\prod_x$ means the second component of that pair is the proof itself. Notice how its type depends on the returned δ : it is a function that takes any real number x , assumes $|x - 2| < \delta$, and returns a proof of the required estimate $|x^2 - 4| < \epsilon$.

Proving the continuity statement is the same as giving a term of this type.

2.3. Giving a term

In the notation of λ -calculus, the proof has the following term-shaped skeleton:

$$\text{sqContAt2} := \lambda\epsilon. \text{let } \delta := \min(1, \epsilon/5) \text{ in} \\ \langle \delta, \lambda x. \lambda h_x. h_{\text{estimate}} \rangle \quad (5)$$

where:

$$h_x : |x - 2| < \delta, \quad h_{\text{estimate}} : |x^2 - 4| < \epsilon$$

Thus the proof is a function from tolerances ϵ to certified choices of tolerances δ . To understand how this connects to the manual proof, consider how h_{estimate} is built. In a formal proof assistant, the ordinary chain of inequalities from our example box translates into a sequence of function applications. For instance, we might have a function of the form

$$x < y \longrightarrow y < z \longrightarrow x < z$$

which sends a proof that $x < y$ to a function which sends a proof of $y < z$ to a proof of $x < z$. This corresponds to transitivity of "less than". Each logical step; such as factoring a polynomial, or applying this transitivity, corresponds to feeding some assumption into a pre-proven mathematical lemma. By composing these functions together, the program computes the final proof term h_{estimate} , and the kernel can easily check it has the desired type since it knows the type of each function we used.

3. Constructive mathematics

The Curry–Howard correspondence has some important practical implications with regard to other systems of formal logic. This sort of logical framework can only express *constructive proof*. Statements like IVT:

for any continuous $f : [a, b] \rightarrow \mathbb{R}$ where $f(a) < 0$ and $f(b) > 0$, exists some $c \in [a, b]$ s.t. $f(c) = 0$

cannot be proven without introducing something like the *law of excluded middle* (every statement is true or false). This is because there is no algorithm which can compute the "intermediate value" for *any* function f .

This is prohibitive for doing formal mathematics. But it also makes the Curry–Howard correspondence very explicit. For instance, if I constructively prove that every number is even or odd, I have implicitly specified a computer program which determines whether any given number is even or odd. Moreover, in many proof assistants, I could automatically generate this computer code in a process called *extraction*.

Different proof assistants take different stances on this topic. For instance Rocq by default does not have the law of excluded middle, however you can assert it as an axiom. Lean on the other hand, being built for mathematics, has three core axioms:

- propositional extensionality
- the axiom of choice
- quotient soundness

of which the last two are nonconstructive. These are enough to prove all the desired statements one wishes for in classical logic. Unfortunately, it is impossible to extract a complete computer program from a nonconstructive proof.

4. The rules of dependent type theory

We saw in the above example that we required functions such that the type of the output depends on the specific input. Dependent type theory is the framework that makes this work.

A dependent type theory is organized around judgments made in a context Γ :

$$\Gamma \vdash A : \text{Type} \quad \text{and} \quad \Gamma \vdash a : A$$

The first says that A is a well-formed type; the second says that a is a term of type A . A context is a list of typed variables, and later types may refer to earlier variables:

$$\Gamma = (A : \text{Type}, x : A, B : A \rightarrow \text{Type}, y : B(x))$$

4.1. Dependent function types

If $B(x)$ is a type for every $x : A$, then

$$\prod_{x:A} B(x)$$

is the type of functions f whose output $f(x)$ has type $B(x)$. The codomain may change with the input. The following are the formal rules that these types obey:

Formation, introduction, elimination, and computation for Π

$$\frac{\Gamma \vdash A : \text{Type} \quad \Gamma, x : A \vdash B(x) : \text{Type}}{\Gamma \vdash \prod_{x:A} B(x) : \text{Type}} \quad (\Pi\text{-form})$$

$$\frac{\Gamma, x : A \vdash b(x) : B(x)}{\Gamma \vdash \lambda x. b(x) : \prod_{x:A} B(x)} \quad (\Pi\text{-intro})$$

$$\frac{\Gamma \vdash f : \prod_{x:A} B(x) \quad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B(a)} \quad (\Pi\text{-elim})$$

$$(\lambda x. b(x))(a) \equiv b(a) \quad (\Pi\text{-computation, or } \beta\text{-rule})$$

If B does not depend on x , then $\prod_{x:A} B$ is just the ordinary function type $A \rightarrow B$. Logically, $\prod_{x:A} P(x)$ expresses $\forall x : A, P(x)$.

The symbol $\equiv$ denotes *definitional equality*: two expressions compute to the same normal form, such as $(\lambda x. b)(a) \equiv b[a/x]$. Effectively, there are many simple rules; for instance if f has type $A \rightarrow B$ and x has type A , then $f(x)$ has type B . Two expressions are definitionally equal if they can be related to each other via these rules. The kernel checks definitional equality, and type checking uses definitional equality constantly. The type $\text{Id}_A(a, b)$ we will see later represents *propositional equality*, which requires a proof term.

5. Inductive types

Inductive types are the core mechanism by which mathematical objects are defined and data is reasoned about. An inductive type is specified by constructors which are the rules by which one may construct terms of that type.

5.1. Natural Numbers

For the natural numbers, the formation and constructor rules are:

$$\mathbb{N} : \text{Type}, \quad \text{zero} : \mathbb{N}, \quad \text{succ} : \mathbb{N} \rightarrow \mathbb{N}$$

The kernel automatically puts a term of the following type, called the *eliminator* into the context:

$$\text{Nat.ind} : \prod_{P : \mathbb{N} \rightarrow \text{Type}} P(\text{zero}) \rightarrow \left(\prod_{n : \mathbb{N}} P(n) \rightarrow P(\text{succ}(n)) \right) \rightarrow \prod_{n : \mathbb{N}} P(n) \quad (6)$$

Here P is the *motive*: a family of types indexed by n . Under Curry–Howard, if each $P(n)$ is a proposition, (6) is mathematical induction.

The computation rules say that induction computes on constructors:

$$\begin{aligned} \text{Nat.ind}(P, z, s, \text{zero}) &\equiv z, \\ \text{Nat.ind}(P, z, s, \text{succ}(n)) &\equiv s(n, \text{Nat.ind}(P, z, s, n)) \end{aligned}$$

These are part of the meaning of $\mathbb{N}$, not additional theorems about it. The intuition is that in order to map out of an inductive type, one needs to specify how the function unwraps the constructors, called *elimination*. For instance $f(x) = x^2$ can be defined by

$$f(0) \equiv 0 \quad f(\text{succ}(n)) \equiv f(n) + 2n + 1$$

assuming I have defined addition (coming up).

5.2. Equality

One of the most important instances of inductive types is the concept of *propositional equality*. In Lean-like notation

$$\begin{aligned} \text{Inductive } \text{Id } \{A : \text{Type}\} (a : A) : A \rightarrow \text{Type} := \\ | \text{refl}_a : \text{Id } \{A\} a a \end{aligned}$$

so after choosing some type A and $a : A$, the inductive $\text{Id}_{A,a}$ has a single constructor refl_a which takes two definitionally equal elements and constructs an instance of this type. We say that $x : A$ and $y : A$ are *propositionally equal* if we can construct a term of type $\text{Id}_A(x, y)$. This is different than definitional equality since it can and must be proven. The axioms in section 3 are formulated as trusted functions which construct terms of this type. We can use this notion to prove some theorems.

5.3. Addition by zero

Define addition recursively on the first argument:

$$\text{zero} + n \equiv n \quad \text{succ}(m) + n \equiv \text{succ}(m + n)$$

I want to prove $n + \text{zero} = n$ (these are not definitionally equal since we have put the zero on the other side). The statement $n + \text{zero} = n$ has type:

$$\prod_{n : \mathbb{N}} \text{Id}_{\mathbb{N}}(n + \text{zero}, n)$$

It is proved using Nat.ind with motive:

$$P(n) := \text{Id}_{\mathbb{N}}(n + \text{zero}, n)$$

The base case is $\text{refl}_{\text{zero}}$. In the step, a proof $p : m + \text{zero} = m$ is sent to the equality

$$\text{succ}(m + \text{zero}) = \text{succ}(m)$$

obtained by applying succ to both sides of p . Thus recursion constructs data, while dependent recursion constructs proofs about that data.

5.4. Path induction

You might guess that the eliminator for the identity is quite important. It often called J or path induction, it formalizes substitution of equals for equals:

$$\text{Id.ind}_{A,a} : \prod_{P : (\prod_{x:A} \text{Id}_A(a,x) \rightarrow \text{Type})} \left(P(a, \text{refl}_a) \longrightarrow \prod_{x:A} \prod_{p:\text{Id}_A(a,x)} P(x, p) \right)$$

It is what lets a proof assistant transport data and proofs across an equality. For instance, suppose I want to prove that if $x, y : A$ are propositionally equal, and T is a family of propositions indexed by A , then $T(x)$ implies $T(y)$. I desire to construct a term of type:

$$\prod_{x,y:A} \text{Id}_A(x, y) \longrightarrow T(x) \longrightarrow T(y) \tag{7}$$

I need to map out of an inductive type, so it's time to use the eliminator. Define the motive P as the function type that maps any endpoint z to $T(x) \rightarrow T(z)$:

$$P : \lambda(z : A)(q : \text{ld}_A(x, z)).(T(x) \rightarrow T(z))$$

By feeding this into $\text{ld.ind}_{A,x}$ I get a term of type:

$$\text{ld.ind}_{A,x}(P) : P(x, \text{refl}_x) \longrightarrow \prod_{z:A} \prod_{p:\text{ld}_A(x,z)} P(z, p)$$

Notice that $P(x, \text{refl}_x) \equiv P(x) \rightarrow P(x)$. The identity function is a term of this type, so feeding it into our expression gives a term of type:

$$\prod_{z:A} \prod_{p:\text{ld}_A(x,z)} T(x) \rightarrow T(z)$$

Plugging in y for z gives a term of type

$$\prod_{p:\text{ld}_A(x,y)} T(x) \rightarrow T(y)$$

and abstracting over x, y and $p : \text{ld}_A(x, y)$ constructs a term of the desired type in (7). It is quite remarkable that the eliminator the kernel automatically generates when we define this specific inductive type gives us the machinery we need to perform this type of logical deduction. One would expect for induction to be the most primitive form of logical argument.

6. More Inductives

You may have noticed that the ld inductive type was actually a large family of inductive types (despite my attempt to sweep it under the rug). I.E. for any type A and $a : A$ we have a type $\text{ld}_{A,a}$. In the kernel, this is represented by a dependent function type mapping to an inductive type whose definition depends on the specific input. The list is one of the simplest examples of this concept.

6.1. Lists and Vectors

Fix a type $A : \text{Type}$. The type of finite lists with entries in A is generated by:

$$\text{List}(A) : \text{Type} \quad \text{nil} : \text{List}(A) \quad \text{cons} : A \rightarrow \text{List}(A) \rightarrow \text{List}(A)$$

Its dependent induction principle is:

$$\begin{aligned} \text{List.ind}_A : & \prod_{P:\text{List}(A) \rightarrow \text{Type}} P(\text{nil}) \\ & \rightarrow \left(\prod_{a:A} \prod_{\ell:\text{List}(A)} P(\ell) \rightarrow P(\text{cons}(a, \ell)) \right) \rightarrow \prod_{\ell:\text{List}(A)} P(\ell) \end{aligned} \tag{8}$$

To define a function out of lists, specify what it does to nil and to $\text{cons}(a, \ell)$, assuming its value is already known on ℓ .

For example, length is defined by:

$$\text{length}(\text{nil}) \equiv \text{zero} \quad \text{length}(\text{cons}(a, \ell)) \equiv \text{succ}(\text{length}(\ell))$$

Likewise, for $f : A \rightarrow B$, the map function has type

$$\text{map} : (A \rightarrow B) \rightarrow \text{List}(A) \rightarrow \text{List}(B)$$

and is defined constructor by constructor.

6.2. Where the dependence appears

Once A is fixed, $\text{List}(A)$ is an ordinary inductive type. Nevertheless, dependent type theory appears in two places:

- the list constructor itself is polymorphic, $\text{List} : \text{Type} \rightarrow \text{Type}$;
- the eliminator (8) is dependent because its target is the varying family $P(\ell)$.

A more visibly dependent list-like object records its length in its type. Define vectors by the indexed family

$$\text{Vec}(A) : \mathbb{N} \rightarrow \text{Type}$$

with constructors:

$$\begin{aligned} \text{vnil} &: \text{Vec}(A)(\text{zero}) \\ \text{vcons} &: \prod_{n:\mathbb{N}} A \rightarrow \text{Vec}(A)(n) \rightarrow \text{Vec}(A)(\text{succ}(n)) \end{aligned}$$

Now the index changes as constructors are applied. The type system can rule out impossible cases. For example,

$$\text{head} : \prod_{n:\mathbb{N}} \text{Vec}(A)(\text{succ}(n)) \rightarrow A$$

needs no empty-vector branch: its input type proves that the vector has positive length. Similarly, vector append can state its exact size law in its type:

$$\text{append} : \prod_{m,n:\mathbb{N}} \text{Vec}(A)(m) \rightarrow \text{Vec}(A)(n) \rightarrow \text{Vec}(A)(m + n)$$

The program and its size invariant are checked simultaneously.

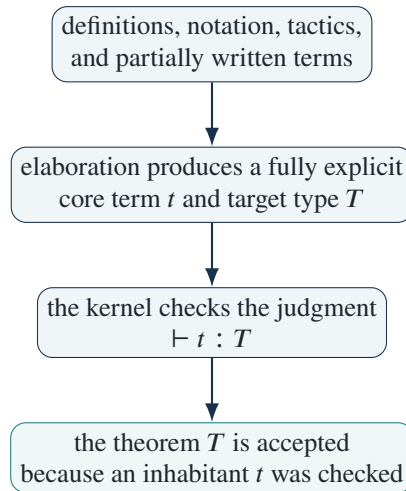
Lists versus vectors

	Type	What the type checker knows
List	$\text{List}(A)$	entries have type A ; length is runtime data
Vector	$\text{Vec}(A)(n)$	entries have type A and the length is n

A vector is not a list plus a comment saying what its length should be. Its length is part of the type, so a mismatch becomes a typing error rather than a failed test.

7. What proof verification actually checks

A modern dependent-type proof assistant usually separates convenient user-facing tools from a small trusted core. The workflow is



During type checking, the kernel recursively applies rules such as those for Π , Σ , identity types, and inductive eliminators. It also computes when comparing types:

$$(\lambda x.t)(a) \equiv t[a/x] \quad \text{Nat.ind}(P, z, s, \text{zero}) \equiv z$$

and similarly for the constructors of lists. Systems such as Rocq and Lean follow the general design in which high-level proof automation produces proof terms that are checked again by the trusted kernel. The kernel is intentionally small and compact (about 5,000 lines of C++ for Lean), to reduce the chance of bugs.

For the continuity example, the kernel does not decide continuity by numerical testing. It checks that

- the program accepts every positive ε ;
- the returned first component is a real δ ;
- the second component proves $0 < \delta$;
- the final function turns arbitrary x and a proof of $|x - 2| < \delta$ into a proof of $|x^2 - 4| < \varepsilon$.

Every use of algebra, order, absolute values, and min must itself be justified by previously verified lemmas or declared axioms.

The limits

A proof assistant checks that a formal term inhabits a formal type relative to the systems rules. It does not guarantee that the formal statement is the statement the author intended. Moreover, it does not prove the theorem for you (even though proof automation tools are rapidly becoming more powerful).

The *de Bruijn factor* is the average ratio of the length of a machine-checkable proof and a human-readable proof of the same statement. Lean is roughly estimated to have a de Bruijn factor somewhere between 1 and 4. Microsoft Research claims that the "formal mathematics revolution" will be complete when Lean reaches a de Bruijn factor of 1. Only time will tell if this happens.

8. Conclusion

If one remembers anything from reading these notes, remember the following three concepts:

Curry–Howard: proving a proposition means constructing a term of the corresponding type.

Dependent types: Π -types express universally quantified functions.

Inductive types: constructors generate data, and dependent eliminators generate programs and proofs by recursion and induction.